

Week 5 demo — fallback captures

Bike-sharing component diagram

Captured with Claude Code, Sonnet 5 (low effort), September 2026

When to use this deck

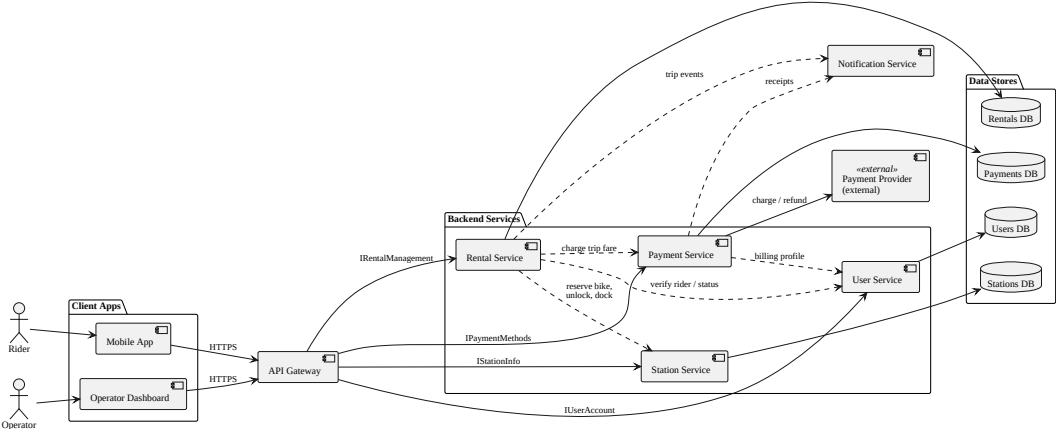
Switch here if the live AI fails. These are real answers to the demo's own prompts; walk the same defect catalogue against them.

Prompt #1

“Generate a UML component diagram (as PlantUML) for the city bike-sharing app from last week: rentals, stations, payments, and users.”

AI's answer — the diagram

City Bike-Sharing App - Component Diagram



AI's answer — what it said

I don't have last week's session or any code for that app, so this diagram is built only from the four areas you named. Component names and interfaces are my assumptions. If you paste the actual design, I'll align it.

[...]

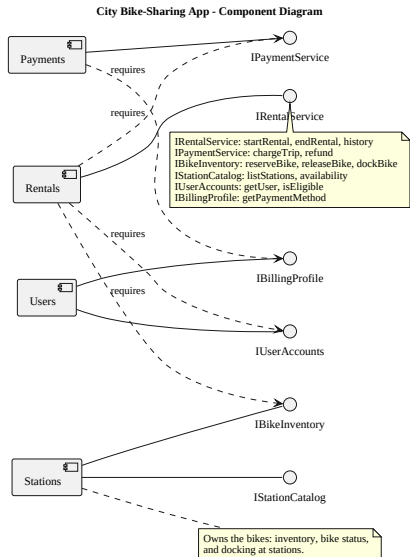
Assumptions:

- ▶ *The services are split by domain, and each service owns its own database.*
- ▶ *The payment provider is an external system.*
- ▶ *I added an API gateway and a notification service. Remove them if your app doesn't have them.*

Prompt #2 (same session)

“Revise it. Drop any infrastructure the prompt didn’t mention — no API gateway, web portal, notification service, or per-service databases. Group by domain capability without nested sub-components — aim for a handful of parts, and say which part owns the bikes. Each component must declare the interfaces it provides and requires. Dependencies must point one way, with no cycles.”

AI's answer — the revised diagram



Al's answer — its design notes

Design:

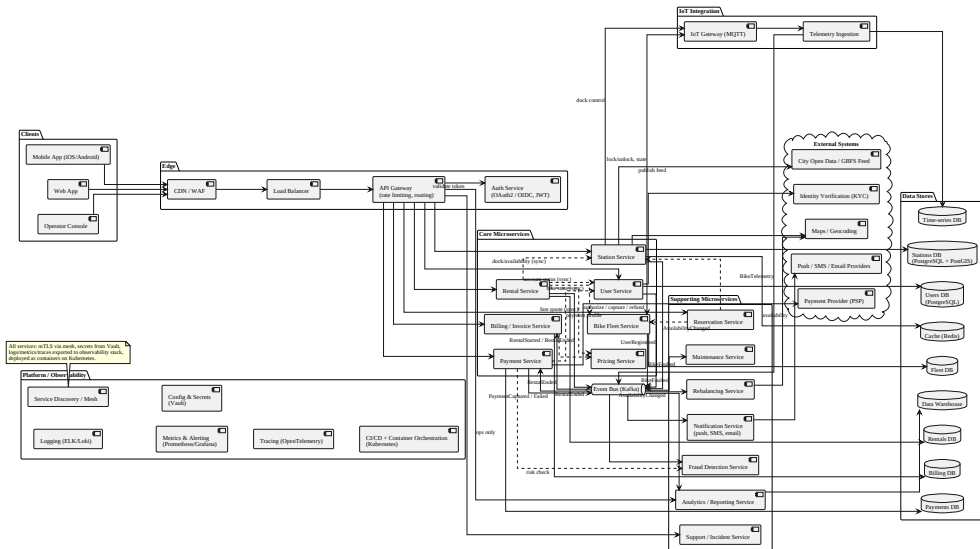
- ▶ **Stations owns the bikes.** *It holds their inventory, status, and docking. Rentals must call `IBikeInventory` to reserve, release, or dock a bike.*
- ▶ **Four components, no nesting.**
- ▶ *[...]*
- ▶ **No cycles.** *The dependencies run in one direction: `Rentals` → `Payments` → `Users`, with `Rentals` also → `Stations` and → `Users`. `Stations` and `Users` are leaves.*
- ▶ **Rider-facing interfaces have no consumer.** *`IRentalService` and `IStationCatalog` aren't required by any component in the diagram. That's intentional, since I left out clients like a web portal or gateway.*

Make-it-fail reserve (fresh session, after prompt #1)

“Make this production-ready with all the supporting infrastructure and microservices.”

AI's answer — the “production-ready” diagram

City Bike-Sharing Platform - Production Component Diagram



AI's answer — what it said

- ▶ **Edge:** a CDN/WAF, load balancer, API gateway and a dedicated auth service.
- ▶ **Supporting services:** Reservation, Notification, Maintenance, Rebalancing, Analytics, Fraud Detection and Support.
- ▶ **Communication:** [...] Everything else goes through a Kafka event bus [...]
- ▶ **Platform:** service mesh with mTLS, Vault, centralized logs, metrics and tracing, and Kubernetes.

Caveats:

- ▶ The technology choices (Kafka, PostgreSQL, Vault, Kubernetes and so on) are reasonable defaults I picked, not requirements. Swap in whatever your stack uses.
- ▶ [...] I haven't rendered it, so *linetype ortho* with this many edges may look cluttered.